To send an email in an Android app, you can use Intents to open an email client (like Gmail) with the email pre-filled. However, to automatically send an email without user interaction, you'll need to use an SMTP (Simple Mail Transfer Protocol) library like JavaMail.

Approach 1: Using an Email Intent (Recommended for Simplicity)

This is the most common and simplest approach. The email client (such as Gmail) will be opened with the fields pre-filled, and the user has to manually send the email.

Example Code:

```java
Intent intent = new Intent(Intent.ACTION_SEND);
intent.setType("message/rfc822");
intent.putExtra(Intent.EXTRA_EMAIL, new String[]{"recipient@example.com"});
intent.putExtra(Intent.EXTRA_SUBJECT, "Subject of the email");
intent.putExtra(Intent.EXTRA_TEXT, "Body of the email");

try {
    startActivity(Intent.createChooser(intent, "Send Email"));
} catch (android.content.ActivityNotFoundException ex) {
    Toast.makeText(MainActivity.this, "No email client is installed.",
Toast.LENGTH_SHORT).show();
}
```

- Intent.ACTION_SEND: This tells Android you want to send something.
- setType("message/rfc822"): This specifies the MIME type for email.
- Intent.EXTRA_EMAIL: Add recipients.
- Intent.EXTRA_SUBJECT: Set the email subject.
- Intent.EXTRA_TEXT: Set the email body.

Advantages:
- Very simple to implement.
- No need to handle the complexities of SMTP or user credentials.

Disadvantages:
- The user needs to manually send the email via an external email app.
- Cannot be used for background email sending.

Approach 2: Sending Email Programmatically via JavaMail (for Auto-Sending)

To send an email directly from your app without user interaction, you'll need the JavaMail API to communicate with Gmail's SMTP server.

Steps:

1. Add the JavaMail Library:
   - Add JavaMail to your project's `build.gradle` file:

   ```groovy
   implementation 'com.sun.mail:android-mail:1.6.7'
   implementation 'com.sun.mail:android-activation:1.6.7'
   ```

2. Request Internet Permission:
   - Add the following permission to your `AndroidManifest.xml`:

   ```xml
   <uses-permission android:name="android.permission.INTERNET"/>
   ```

3. Write JavaMail Code:

   Here's how you can use the JavaMail API to send an email programmatically:

   ```java
   import java.util.Properties;
   import javax.mail.Authenticator;
   import javax.mail.Message;
   import javax.mail.PasswordAuthentication;
   import javax.mail.Session;
   import javax.mail.Transport;
   import javax.mail.internet.InternetAddress;
   import javax.mail.internet.MimeMessage;

   public class MailSender {

       public static void sendEmail(final String username, final String password, String toEmail,
String subject, String body) {
           Properties props = new Properties();
           props.put("mail.smtp.host", "smtp.gmail.com");
           props.put("mail.smtp.socketFactory.port", "465");
           props.put("mail.smtp.socketFactory.class", "javax.net.ssl.SSLSocketFactory");
           props.put("mail.smtp.auth", "true");
           props.put("mail.smtp.port", "465");

           Session session = Session.getDefaultInstance(props, new Authenticator() {
   ```

```
            @Override
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication(username, password);
            }
        });

        try {
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(username));
            message.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(toEmail));
            message.setSubject(subject);
            message.setText(body);

            // Send the message
            Transport.send(message);
            System.out.println("Email Sent!");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

- `username`: Your Gmail address (e.g., `yourname@gmail.com`).
- `password`: Your Gmail password or App-Specific password (for Gmail security, regular passwords are often blocked, so you may need to set up App-Specific Passwords in Gmail settings).
- `toEmail`: Recipient's email.
- `subject`: Email subject.
- `body`: Email body content.

4. Call the `sendEmail()` Method:
In your activity or where needed, call the `sendEmail()` method:

```java
MailSender.sendEmail("yourname@gmail.com", "yourpassword",
"recipient@example.com", "Test Subject", "Hello, this is a test email!");
```

Important Notes:
- Gmail App-Specific Password: If you use Gmail, you might need to generate an App-Specific Password from your Google Account if the normal password is blocked due to security settings.
- Go to [Google Account Settings](https://myaccount.google.com/security) → "Signing in to Google" → "App passwords" to generate one.

- Security: Be cautious about storing sensitive information (like passwords) in the app. It’s better to use a server-side solution or encrypted storage for production apps.

Advantages:
- No user interaction required to send the email.
- Can be used for background tasks like sending emails in a service.

Disadvantages:
- Requires handling Gmail security restrictions.
- Less secure for sensitive data, especially passwords in the app.
- May not work if Google flags the request as insecure.

Which to Choose?

- If simplicity is the goal: Use the Intent-based approach (Approach 1). It's quick, easy, and leverages existing email clients (like Gmail or Outlook).
- If automatic email sending is needed: Use JavaMail with Approach 2. However, be cautious about storing email credentials and handling security concerns.